

# Input binding

Why our Dropdown takes value and onChange instead of just remembering the answer itself

useState

props down

events up

# The browser already keeps its own copy

Type into a plain `<input>` and the value lives in the DOM. Your component has no idea it changed, so it cannot filter, validate, reset or even read it without going and asking.

## Uncontrolled

```
<input type="text" />
```

The DOM holds the text. React renders the box once and looks away.

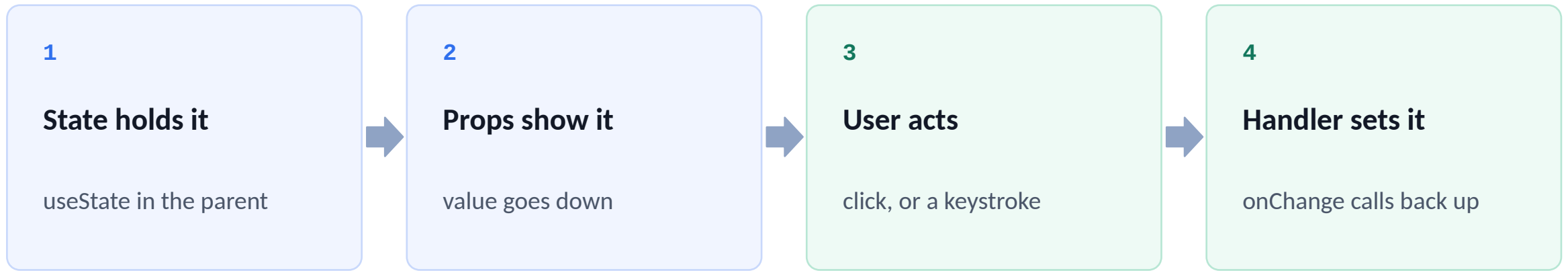
## Controlled

```
<input value={term}  
  onChange={handleChange} />
```

State holds the text. The box only ever shows what state says.

Two copies of the same fact is a bug waiting to happen. Pick one, and make it state.

# Binding is a loop with four steps



Step 4 changes state, which starts step 1 again — so what you see on screen is always what state says. The component in the middle never decides anything; it displays and it reports.

State goes down as props • events come back up as function calls

# Our Dropdown, both halves

## DropdownPage — owns it

```
const [value, setValue] =
  useState(null)

const handleChange = (option) => {
  setValue(option)
}

<Dropdown
  options={OPTIONS}
  value={value}
  onChange={handleChange} />
```

## Dropdown — displays and reports

```
const {options, value,
  onChange} = props

const handleOptionClick = (option) => {
  setIsOpen(false)
  onChange(option)
}

{value ? value.label : 'Select...'}
```

No useState for the selection inside Dropdown. It reads value to know what to show, and calls onChange to say what the user did.

# What owning the value buys the page

## Filter a list

```
DATA.filter(s =>
  s.team === value.value)
```

The roster on our page reacts because the page, not the Dropdown, knows the answer.

## Drive anything else

```
COLOR_MAP[value?.value]
```

One selection colours the heading, sets a chart, hides a section — all from one variable.

## Set it from outside

```
setValue(null)
```

A Reset button, a saved draft, a value from an API — the box follows state wherever it comes from.

Same trade next class: the Modal does not own modalOpen either — the page does, because the button and the modal both need it.

# Four things to remember

## They come as a pair

A value with no `onChange` is a box nobody can change. React warns you in the console.

## Ours hands back an object

Our Dropdown calls `onChange(option)` — the whole `{label, value}`. You decide what your component reports.

## For real inputs, read the event

```
onChange={(e) =>
  setTerm(e.target.value)}
```

## Start it at a real value

`useState(null)` for us, `useState('')` for text — never undefined, or the input starts uncontrolled and switches later.

**The question to ask every time:** who else needs to know this? That answers where the state lives, and the rest is just value down, `onChange` up.